Лабораторная работа № 18: Исследование методов работы модуля OS.

Цель работы: Сформировать умение анализировать методы работы модуля OS.

Задание 1 (Базовый уровень)

Получение базовой информации о платформе и архитектуре

Описание:
Необходимо создать Node.js-скрипт, который выводит в консоль базовую информацию об операционной системе с помощью методов модуля `os`.

Как это должно работать:

1. Скрипт запускается командой в терминале
2. Выводится информация об операционной системе:
    - Идентификатор платформы (`os.platform()`)
    - Человекочитаемое имя ОС (`os.type()`)
    - Разрядность процессора (`os.arch()`)
    - Версия ОС (`os.release()`)
    - Имя хоста в сети (`os.hostname()`)
    - Время работы системы в секундах (`os.uptime()`)
3. На основе полученной платформы скрипт делает вывод:
    - Для `win32` — "Вы работаете в Windows"
    - Для `linux` — "Вы работаете в Linux"
    - Для `darwin` — "Вы работаете в macOS"
    - Для остальных — "Неизвестная платформа"

Пример работы:

```
text
```

```
$ node os-info.js

=== Информация о системе (группа ББМО-01-23) ===
Платформа:      win32
Тип ОС:         Windows_NT
Архитектура:    x64
Версия ОС:      10.0.19045
Имя хоста:      STUDENT-PC
Время работы:   3 ч 25 мин 12 сек

Вы работаете в Windows
```

Что нужно сделать для отчёта:

- Сделать скриншот запуска скрипта в терминале
- Сделать скриншот вывода на вашей ОС (Windows/Linux/macOS)
- Показать определение платформы (условный вывод)

Задание 2 (Средний уровень)

Информация о процессоре и памяти

Описание:
Необходимо расширить скрипт для сбора информации о процессоре и оперативной памяти.

Как это должно работать:

1. Информация о процессоре:
    - Общее количество логических ядер
    - Модель процессора
    - Частота каждого ядра в МГц
    - Расчёт средней частоты всех ядер
2. Информация о памяти:
    - Общий объём памяти в гигабайтах
    - Свободная память в гигабайтах
    - Процент использования памяти
3. Средняя загрузка системы:
    - Массив из трёх значений: за 1, 5 и 15 минут
    - Для Windows — пометка о недоступности
4. Адаптивный вывод:
    - Если свободной памяти меньше 20% — предупреждение
    - Для группы ББМО-01-23 — строка с номером группы

Пример работы:

```
text
```

```
$ node os-resources.js

=== Информация о процессоре ===
Количество логических ядер: 8
Модель процессора:          Intel(R) Core(TM) i5-8250U
Средняя частота:            1800 МГц

=== Информация о памяти ===
Общий объём:                15.89 ГБ
```

Свободно: 8.23 ГБ
Использовано: 7.66 ГБ (48.2%)

Группа: ББМО-01-23

Память в норме

Что нужно сделать для отчёта:

- Скриншот вывода информации о процессоре
- Скриншот вывода информации о памяти
- Скриншот вывода средней загрузки
- Скриншот предупреждения при малом объёме памяти

Задание 3 (Продвинутый уровень)

Сетевые интерфейсы и информация о пользователе

Описание:
Реализовать скрипт для получения информации о сетевых интерфейсах и текущем пользователе.

Как это должно работать:

1. Информация о сетевых интерфейсах:
    - Список всех интерфейсов
    - IP-адрес (IPv4 и IPv6), MAC-адрес
    - Определение "основного" интерфейса (первый внешний IPv4)
2. Информация о пользователе:
    - Имя пользователя
    - UID и GID (для UNIX)
    - Домашняя директория
    - Оболочка по умолчанию
3. Безопасность:
    - MAC-адреса маскируются (например, `AA:BB:CC:**:**:**`)
    - Для группы ББМО-01-23 — проверка, что пользователь не root

Пример работы:

```
text
```

$ node os-network.js

=== Сетевые интерфейсы ===
Интерфейс: Ethernet

```
  IPv4:        192.168.1.105
  MAC:         AA:BB:CC:**:**:**
  Внутренний:  нет

Всего интерфейсов:        3
Основной интерфейс:       Ethernet (192.168.1.105)

=== Информация о пользователе ===
Имя пользователя:         student
UID:                      1000
Домашняя директория:      /home/student

Группа:                   ББМО-01-23

Проверка root:            нет
```

Что нужно сделать для отчёта:

- Скриншот списка сетевых интерфейсов
- Скриншот информации о пользователе
- Скриншот маскирования MAC-адресов
- Скриншот проверки root-пользователя

Задание 4 (Сложный уровень)

Мониторинг системы в реальном времени

Описание:
Разработать скрипт, который собирает "слепок" системы и периодически обновляет его, отслеживая изменения ключевых показателей.

Как это должно работать:

1. Первичный сбор данных:
    - Все данные из предыдущих заданий собираются в один объект
    - Объект имеет метку времени и номер группы
2. Расчёт загрузки CPU за интервал:
    - Два замера `os.cpus()` с интервалом
    - Расчёт разницы в полях `times`
    - Формула: `загрузка = busy_diff / total_diff * 100`
3. Периодический мониторинг:
    - Обновление каждые 2 секунды
    - Перерисовка через `\r`
    - Выход по Ctrl+C
4. Адаптивное поведение:

    - CPU > 80% — критично
    - CPU > 50% — предупреждение
    - Иначе — норма
    - Память < 10% свободно — предупреждение
5. Логирование:
    - Аномалии пишутся в `monitor.log`
    - Формат: `[YYYY-MM-DD HH:MM:SS] [ГРУППА] Предупреждение: ...`

Пример работы:

```
text
```

```
$ node os-monitor.js

Мониторинг (группа ББМО-01-23). Ctrl+C для выхода.

CPU: 12.5% | RAM: 48.2% (8.23 ГБ свободно)
CPU: 78.9% | RAM: 52.1% (7.61 ГБ свободно)  ⚠
CPU: 92.4% | RAM: 55.3% (7.10 ГБ свободно)  ⚠⚠
^C
```

```
Мониторинг остановлен. Предупреждений: 2
```

Что нужно сделать для отчёта:

- Скриншот первичного сбора данных
- Скриншот работы мониторинга
- Скриншот расчёта загрузки CPU
- Скриншот содержимого `monitor.log`
- Скриншот корректного завершения

Фиксация изменений

Добавить файлы в Git
Создать коммит
Отправить в удалённый репозиторий
Создать Pull Request